

COMPTE RENDU : SIMULATION MÉTÉO DISTRIBUÉE

ACCÉLÉRÉE SUR GPU

1. Objectifs et Présentation du Projet

Ce projet implémente un système de simulation de **diffusion thermique 2D** (utilisé comme moteur pour des prévisions climatiques). L'objectif principal est de concevoir une architecture distribuée et hautement performante reposant sur trois piliers technologiques :

1. **Partitionnement de domaine** : La grille météo globale est découpée en bandes horizontales réparties entre différents processus.
2. **Accélération matérielle** : Le calcul du stencil de diffusion locale est déporté sur les cœurs CUDA du GPU via la bibliothèque **CuPy**.
3. **Calcul distribué (MPI)** : La synchronisation et la cohérence physique aux jointures des sous-grilles sont assurées par l'échange de lignes fantômes (**halos**) via **mpi4py**.

2. Architecture Logicielle et Modules du Projet

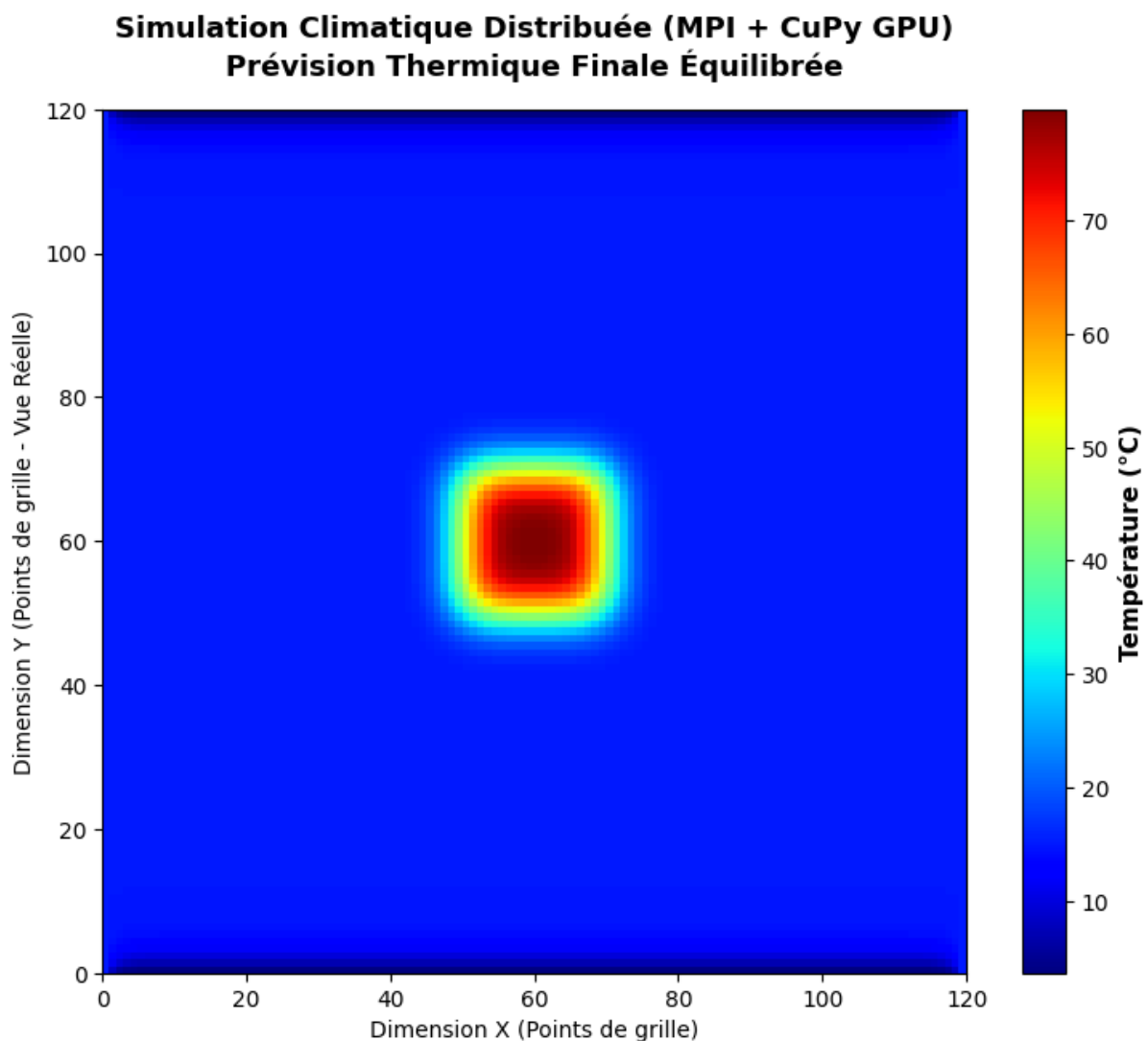
Le code est structuré de manière modulaire au sein du répertoire `src/` :

- **mpi_manager.py (Infrastructure MPI)** : Gère la topologie du système. Il détermine le rang de chaque processus, identifie ses voisins directs (haut/bas), découpe la grille globale (`scatter_grid`) et reconstruit la grille finale sur le nœud maître (`gather_grid`).
- **simulation_kernel.py (Noyau GPU)** : Applique l'équation de diffusion thermique via un stencil 2D (prise en compte des 4 voisins directs). Le calcul s'exécute en parallèle sur le GPU en utilisant des tableaux CuPy (`cp.copy`, `cp.array`).
- **halo_exchange.py (Échange des frontières)** : Ajoute deux lignes virtuelles (**halos**) autour de la bande de données locale. Il utilise l'opération synchrone et

bidirectionnelle Sendrecv de MPI pour transférer les frontières entre voisins directs en évitant les risques d'interblocage (*deadlocks*).

- **main_simulation.py (Orchestrateur)** : Coordonne le workflow à chaque pas de temps (Échange MPI → Copie CPU-GPU → Calcul de diffusion sur GPU → Extraction de la zone utile). À la fin des 50 itérations, le nœud maître réassemble la grille et la sauvegarde sous `outputs_final_grid.npy`.

4. Résultats de la Simulation et Analyse des Performances



A. Validation Scientifique (Visualisation)

Le chargement de la matrice finale (120×120 points) via une Heatmap (matplotlib) confirme la validité du modèle. La chaleur injectée initialement au centre s'est diffusée de manière fluide et continue à travers les frontières des bandes. Cela démontre que **l'échange de halos**

fonctionne correctement et maintient la continuité physique entre les sous-domaines gérés par des processus distincts.

B. Analyse Comparative des Performances (Benchmark)

L'exécution a été mesurée sur une configuration de test pour 50 itérations :

- **Résolution** : Grille de 120×120 points
- **Temps (1 processus - Séquentiel)** : 1,1618 secondes
- **Temps (3 processus - Parallèle)** : 2,1483 secondes
- **Accélération (Speedup)** : $0,54 \times$

C. Interprétation du Speedup (< 1)

Le Speedup obtenu est de $0.54 \times$, ce qui signifie que la version parallèle est *plus lente* que la version séquentielle. Ce phénomène s'explique par deux facteurs critiques :

1. **Surcharge de communication (Overhead)** : La taille de la grille (120×120) est beaucoup trop petite. Le temps requis pour effectuer les échanges de halos via MPI et transférer les données entre la mémoire CPU et la mémoire GPU (cp.array / cp.asnumpy) à chaque itération surpasse largement le temps gagné sur le calcul lui-même.
2. **Sous-utilisation du GPU** : Les processeurs graphiques (GPU) excellent sur des charges de calcul massives (grilles de plusieurs milliers de lignes/colonnes). Ici, les cœurs CUDA n'atteignent pas leur seuil d'efficacité maximale.